

SQL & PostgreSQL

Módulo 3 — DML

Data Manipulation Language

INSERT • UPDATE • DELETE • SELECT • WHERE • Subqueries

Banco: estudodb | Role: devuser | PostgreSQL 18

github.com/Willsmt/estudos-sql-postgresql

Junho 2026

Introdução

Este módulo cobre a DML — Data Manipulation Language, a sublinguagem do SQL responsável por operar sobre os dados armazenados nas estruturas criadas pelo DDL. São abordados INSERT, SELECT com filtros, UPDATE, DELETE, soft delete e subqueries para consultas complexas.

Ambiente utilizado:

- PostgreSQL 18 — Ubuntu 26.04
- Banco: estudodb | Role: devuser
- Acesso via psql (CLI) e DBeaver 26 via SSH Tunnel
- Schemas: blog (5 tabelas) e ecommerce (7 tabelas)

Aula 1 — O que é DML

DML significa Data Manipulation Language — Linguagem de Manipulação de Dados. É a sublinguagem responsável por operar sobre os dados dentro das estruturas já criadas pelo DDL.

Mapa das Sublinguagens SQL

Sublinguagem	Nome Completo	O que faz	Comandos
DDL	Data Definition Language	Define estruturas	CREATE, ALTER, DROP
DML	Data Manipulation Language	Manipula dados	INSERT, UPDATE, DELETE
DQL	Data Query Language	Consulta dados	SELECT
DCL	Data Control Language	Controla acessos	GRANT, REVOKE
TCL	Transaction Control Language	Controla transações	COMMIT, ROLLBACK

Obs: Alguns materiais tratam o SELECT como parte do DML. PostgreSQL e a maioria dos SGBDs modernos o separam em DQL — convenção adotada neste curso.

Os Três Comandos do DML

```
INSERT -- adiciona novos registros

UPDATE -- modifica registros existentes

DELETE -- remove registros (físico)
```

Onde possível, o DELETE físico é substituído pelo soft delete via `is_active = false`, preservando o histórico dos dados.

Aula 2 — Inserir, Buscar e Filtrar

Acessando o banco via psql

```
psql -h localhost -U devuser -d estudodb
```

Nota: -h localhost força conexão TCP/IP, evitando o erro de Peer Authentication quando o usuário do SO difere do role do banco.

Comandos essenciais do psql:

```
\dn          -- lista schemas disponíveis
\dt blog.*    -- lista tabelas do schema blog
\d blog.usuarios -- descreve estrutura da tabela
\x           -- toggle expanded view
\q           -- sair do psql
```

INSERT

A coluna id é omitida — gerada automaticamente por GENERATED ALWAYS AS IDENTITY.

```
INSERT INTO schema.tabela (coluna1, coluna2, ...)
VALUES (valor1, valor2, ...);
```

Múltiplos registros em um único INSERT:

```
INSERT INTO blog.usuarios (username, email, password_hash)
VALUES
    ('willians', 'willians@email.com', 'hash_w'),
    ('maria_silva', 'maria@email.com', 'hash_m'),
    ('joao_dev', 'joao@email.com', 'hash_j');
```

Mais eficiente que múltiplos INSERT individuais — uma só operação no banco.

RETURNING — confirmando o que foi inserido:

```
INSERT INTO blog.tags (name)
VALUES ('postgresql'), ('backend'), ('segurança')
RETURNING id, name;
```

RETURNING retorna os dados do registro recém-criado sem precisar de um SELECT separado. Útil quando o id gerado será usado na operação seguinte.

SELECT

```
-- todas as colunas
SELECT * FROM blog.usuarios;
```

```
-- colunas específicas

SELECT id, username, email FROM blog.usuarios;

-- alias (renomeando colunas no resultado)

SELECT username AS usuario, email AS email_cadastrado FROM blog.usuarios;
```

WHERE — Filtrando dados

```
SELECT * FROM blog.usuarios WHERE username = 'willians';

SELECT id, username FROM blog.usuarios WHERE is_active = true;

SELECT id, username FROM blog.usuarios WHERE username ILIKE 'ana%';

SELECT * FROM blog.tags WHERE name IN ('postgresql', 'backend');

SELECT username, email FROM blog.usuarios WHERE email IS NOT NULL;
```

Operador	Significado
=	Igual
<> ou !=	Diferente
> / <	Maior / Menor
>= / <=	Maior ou igual / Menor ou igual
LIKE	Padrão com curinga %
ILIKE	LIKE case-insensitive
IN	Dentro de uma lista
BETWEEN	Entre dois valores
IS NULL	Valor nulo
IS NOT NULL	Valor não nulo

ORDER BY e LIMIT

```
-- mais recente primeiro

SELECT id, username, created_at FROM blog.usuarios ORDER BY created_at DESC;

-- os 3 primeiros registros

SELECT id, username FROM blog.usuarios ORDER BY id ASC LIMIT 3;
```

Aula 3 — Atualizar e Deletar

UPDATE

```
UPDATE schema.tabela

SET coluna1 = valor1, coluna2 = valor2

WHERE condição;
```

ATENÇÃO: O WHERE é obrigatório. Sem ele, TODOS os registros da tabela são atualizados.

```
-- atualizando um campo

UPDATE blog.usuarios

SET username = 'willians_dev'

WHERE id = 1;

-- múltiplos campos + confirmação via RETURNING

UPDATE blog.usuarios

SET username = 'willians_dev', is_active = true

WHERE id = 1

RETURNING id, username, is_active;
```

Soft Delete — a forma correta de "remover" registros

Em vez de deletar fisicamente, o registro é desativado com `is_active = false`. Garante auditabilidade, rastreabilidade, histórico e possibilidade de reativação futura. Um cadastro desativado continua visível para relatórios e estornos.

```
-- desativar ao invés de deletar

UPDATE blog.usuarios

SET is_active = false

WHERE username = 'ana_tech';

-- verificar todos os registros por status

SELECT id, username, is_active FROM blog.usuarios ORDER BY id;
```

DELETE — Remoção física

```
DELETE FROM schema.tabela WHERE condição;
```

```
-- com RETURNING
```

```
DELETE FROM blog.tags WHERE name = 'linux' RETURNING id, name;
```

ATENÇÃO: DELETE sem WHERE remove TODOS os registros da tabela.

TRUNCATE

```
-- limpa tabela inteira (mais rápido que DELETE sem WHERE)
```

```
TRUNCATE blog.comentarios;
```

```
-- com reinício de sequences (útil para reexecutar seeds)
```

```
TRUNCATE blog.usuarios RESTART IDENTITY CASCADE;
```

TRUNCATE não aceita WHERE. Use apenas em desenvolvimento/teste para limpar tabelas inteiras.

Comportamento das FKs no DELETE

Configuração	Comportamento	Quando usar
ON DELETE CASCADE	Deleta os dependentes junto	Dados que não existem sem o pai
ON DELETE RESTRICT	Bloqueia se houver dependentes	Histórico que precisa ser preservado

```
-- tentativa de deletar usuário com post associado -- FALHA (RESTRICT)
```

```
DELETE FROM blog.usuarios WHERE id = 1;
```

```
-- ERROR: violates foreign key constraint "fk_posts_user_id"
```

```
-- correto: soft delete
```

```
UPDATE blog.usuarios SET is_active = false WHERE id = 1;
```

Aula 4 — Subqueries

Uma subquery é uma consulta dentro de outra consulta. O resultado da query interna é usado como dado de entrada para a query externa. A subquery executa primeiro, depois a query externa usa o resultado.

```
SELECT ... FROM ...  
  
WHERE coluna IN (  
    SELECT coluna FROM outra_tabela WHERE condição -- subquery  
);
```

IN e NOT IN

```
-- posts de usuários ativos  
  
SELECT id, title FROM blog.posts  
WHERE user_id IN (  
    SELECT id FROM blog.usuarios WHERE is_active = true  
);  
  
-- usuários que nunca fizeram post  
  
SELECT id, username FROM blog.usuarios  
WHERE id NOT IN (  
    SELECT DISTINCT user_id FROM blog.posts  
);
```

EXISTS e NOT EXISTS

Mais eficiente que IN quando a subquery retorna muitos registros — o banco para de buscar assim que encontra o primeiro resultado.

```
-- clientes com ao menos um pedido completed  
  
SELECT id, full_name FROM ecommerce.customers c  
WHERE EXISTS (  
    SELECT 1 FROM ecommerce.orders o  
    WHERE o.customer_id = c.id AND o.status = 'completed'  
);
```

O SELECT 1 é convencional — EXISTS só precisa saber se existe algum resultado, não quais são.

Tabela Derivada no FROM

A subquery vira uma tabela temporária consultada pela query externa. Alias é obrigatório.

```
-- produtos com estoque abaixo de 40 unidades

SELECT p.name, estoque.quantity
FROM ecommerce.products p,
     (SELECT product_id, quantity
      FROM ecommerce.stock WHERE quantity < 40) AS estoque
WHERE p.id = estoque.product_id
ORDER BY estoque.quantity ASC;
```

Erro comum: usar o alias da subquery no SELECT sem declará-la no FROM. O alias só existe no escopo onde foi declarado — missing FROM-clause entry for table "alias".

Scalar Subquery no SELECT

Retorna um único valor por linha. Se retornar mais de uma linha, o banco retorna erro.

```
-- total de posts por usuário

SELECT
    u.username,
    (SELECT COUNT(*) FROM blog.posts p
     WHERE p.user_id = u.id) AS total_posts
FROM blog.usuarios u
ORDER BY total_posts DESC;
```

Tipo	Onde aparece	Retorna
IN / NOT IN	WHERE	Lista de valores
EXISTS / NOT EXISTS	WHERE	Verdadeiro ou falso
Tabela derivada	FROM	Conjunto de linhas
Scalar subquery	SELECT	Um único valor por linha
Correlacionada	Qualquer	Depende da linha atual

Boas Práticas e Convenções

- **WHERE é obrigatório em UPDATE e DELETE**

Sem WHERE, a operação afeta todos os registros. Hábito do mercado: escrever o WHERE antes do SET para não esquecer.

- **Soft delete com is_active**

Preferir UPDATE com `is_active = false` ao DELETE físico. Garante auditabilidade, rastreabilidade e possibilidade de reativação.

- **Constraints como regras de negócio**

FKs, CHECK, UNIQUE e NOT NULL são regras de negócio no nível mais baixo. Nenhuma camada acima (aplicação, API, ORM) é tão confiável quanto o banco.

- **RETURNING para confirmar operações**

Use RETURNING após INSERT, UPDATE e DELETE para confirmar o que foi alterado sem precisar de um SELECT separado.

- **Respeitar ordem de INSERT com FKs**

Inserir na ordem: tabelas pai primeiro, dependentes depois. Ex: customers antes de addresses, products antes de stock.

- **NUMERIC(10,2) para valores monetários**

Nunca usar FLOAT — problemas de precisão. Sempre NUMERIC(10,2).

- **EXISTS vs IN em subqueries**

EXISTS é mais eficiente que IN quando a subquery retorna muitos registros, pois para ao encontrar o primeiro resultado.

Referência Rápida

Comandos DML

Comando	Função	Risco sem WHERE
INSERT INTO ... VALUES	Adiciona novos registros	Nenhum
RETURNING	Retorna dados do registro alterado	N/A
UPDATE ... SET	Modifica registros existentes	Atualiza todos
DELETE FROM	Remove fisicamente	Remove todos
TRUNCATE	Limpa tabela inteira	Sempre total
is_active = false	Soft delete — preserva histórico	Nenhum

Filtros e Consultas

Cláusula	Função
WHERE	Filtra quais registros retornam
ILIKE	Padrão case-insensitive com curinga %
IN / NOT IN	Filtra por lista de valores
IS NULL / IS NOT NULL	Verifica presença de valor
ORDER BY ASC/DESC	Ordena o resultado
LIMIT	Limita a quantidade de linhas
AS	Alias para coluna ou tabela derivada

Tipos de Subquery

Tipo	Onde	Retorna	Melhor uso
IN / NOT IN	WHERE	Lista de valores	Listas pequenas
EXISTS / NOT EXISTS	WHERE	Bool (existe ou não)	Listas grandes
Tabela derivada	FROM	Conjunto de linhas	Pré-filtrar dados
Scalar	SELECT	Um valor por linha	Agregar relacionados

Estrutura do Repositório

```
estudos-sql-postgresql/  
├─ schemas/  
|   ├─ blog.sql          -- DDL completo do schema blog  
|   └─ ecommerce.sql     -- DDL completo do schema ecommerce  
├─ seeds/  
|   ├─ blog_seed.sql     -- dados de estudo do schema blog  
|   └─ ecommerce_seed.sql -- dados de estudo do schema ecommerce  
├─ diagrams/  
|   ├─ blog_erd.png  
|   └─ ecommerce_erd.png  
├─ docs/  
|   ├─ modulo1_introducao.pdf  
|   ├─ modulo2_ddl.pdf  
|   └─ modulo3_dml.pdf  
└─ README.md
```